Introducing SALSA

Konstantinos Gkaras & Valentin Oprea & Caolan Leonard (Team 08)
University of Groningen

university of groningen
faculty of science and engineering

Introduction

- **Problem.** State of the art optimization techniques and training methods require users to pick optimal learning rates and learning schedules, a process that costs a lot of time and computational resources. Line search methods can choose these hyperparameters automatically.
- **Gap.** Existing stochastic line searches (SLS) can be combined with state of the art optimizers (SGD, Adam) but ultimately face a number of problems, either by design or due to the disturbances caused by noise. Computational overhead and fluctuations in the step update rule, lead to instabilities and loss of accuracy.
- **Proposal.** SaLSa is a modified Armijo line search criterion, modified with an exponential smoothing term (characterized as "momentum") and an automated mechanism to pick the learning rate depending on a number of optimization criteria. It has been adapted to work both with Stochastic Gradient Descent, and preconditioned optimizers like Adam.
- **Claim.** SaLSA solves accuracy problems due to noise or floating point arithmetic, and on average improves the final loss versus hand-tuned learning rate schedules. It also cuts down on the computational costs (from $\approx 30\%$ to $\approx 3\%$), by identifying and solving a bottleneck of the original line method.

1 Armijo Line Search & Adam Optimizer

1.1 Classic Armijo Line Search

$$f_k(w_k + \eta_k d_k) \leq f_k(w_k) - c \cdot \eta_k ||\nabla f_k(w_k)||^2 \tag{1}$$

This is the decrease condition for the step size η_k, with direction d_k for stochastic mini-batch.

1.2 Modified Adam for Stochastic Mini-Batch

$$\begin{aligned} g_k &= \nabla f_k(w_k) \\ m_k &= \beta_1 \cdot m_{k-1} + (1-\beta_1) \cdot g_k \\ v_k &= \beta_2 \cdot v_{k-1} + (1-\beta_2) \cdot g_k^2 \\ \hat{m}_k &= m_{k-1}/(1-\beta_1^k) \\ \hat{v}_k &= v_{k-1}/(1-\beta_2^k) \\ d_k &= -\hat{m}_k/(\sqrt{\hat{v}_k} + \epsilon) \end{aligned} \tag{2}$$

- g_k the gradient of the $k-$th mini-batch.
- m_k tracks the exponentially weighted moving average of past gradients (a.k.a. the first moment).
- v_k tracks the variance of the gradient estimates $\approx$ the magnitude of the gradient squared (a.k.a. the second moment).
- $\hat{m}_k, \hat{v}_k$ are the bias-corrected estimators of the first and second moments respectively.
- d_k is the preconditioned gradient for Adam, it is divided by the smoothed mean gradient variance for scale normalization.

2 Current State of the Art

Modifying the Armijo line search (1) with the Adam direction (2) yields the Stochastic Line Search (SLS). Adam + SLS performs well on smaller datasets and neural-networks.

2.1 Stochastic Line Search

To modify the Armijo line search criterion so it can be combined with the Adam optimizer, take the momentum $\beta_1 = 0$, and compute the Adam direction according to equation 3.

$$d_k = -\hat{m}_k/(\sqrt{\hat{v}_k} + \epsilon) \tag{3}$$

Replace $||\nabla f_k(w_k)||^2$ in equation (1), by the preconditioned gradient norm

$$\frac{||\nabla f_k(w_k)||^2}{\sqrt{\hat{v}_k} + \epsilon} \tag{4}$$

to yield the Stochastic Line Search criterion

$$f_k(w_k + n_k d_k) \leq f_k(w_k) - c \cdot n_k \frac{||\nabla f_k(w_k)||^2}{\sqrt{\hat{v}_k} + \epsilon} \tag{5}$$

2.2 SLS Failure Cases

The combination of the Adam optimizer and the SLS criterion leads to a numerical scheme that performs well for small datasets and neural-networks. However, it is faced with problems in larger scale training.

Problem 1: Mini-batch Noise

Equations (1) and (5) describe criteria which are checked for every mini-batch. In small mini-batch sizes, these criteria are violated by inherent noise in the data, leading to a wild behavior of the step-size update rule as can be seen by Figure 1.

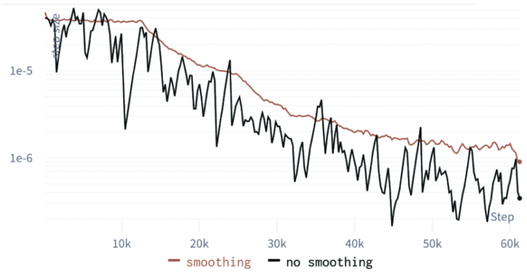

Figure 1: Adam + SLS vs. Adam + SaLSA. Noise in the mini-batch leads to the erratic behavior of the step-size update rule, which is solved by exponential smoothing.

Problem 2: Near-zero mini-batch gradients

In practice, some mini-batches yield gradient estimates so small that $\nabla f_k(w_k) \approx 0$, even with float32 arithmetic enabled. In the original SLS paper, if $\|\nabla f_k(w_k)\| \leq 10^{-8}$, the line search was skipped entirely, as can be seen in Figure 2.

Figure 2: Adam + SLS vs. Adam + SaLSA on the issue of near-zero mini-batch gradients. In the former case, line-search isn't even performed due to computational precision problems, while the latter scheme has no such limitationss.

Problem 3: Algorithmic instability on large datasets

On large datasets, SLS step sizes show high variance and are sensitive to random initialization of the data and shuffling. This leads the algorithm to stall or take very long to converge, as can be seen by Figure 3.

Figure 3: Adam + SLS vs. Adam + SaLSa comparison of variance between runs.

3 Stable Armijo Line Search Adaptation

Tackle mini-batch noise and precision problems by introducing exponential smoothing to the optimizer.

$$\begin{aligned} h_k &= h_{k-1} \cdot \beta_3 + (f_k(w_k) - f_k(w_k + \eta_k d_k)) \cdot (1-\beta_3) \\ s_k &= s_{k-1} \cdot \beta_3 + \|\nabla f_k(w_k)\|^2 \cdot (1-\beta_3) \end{aligned} \tag{6}$$

Yield the Stable Armijo Line Search Adaptation (SaLSa) criterion as:

$$h_k \geq c \cdot \eta_k \cdot s_k \tag{7}$$

Combine it with Adam by modifying s_k:

$$s_k = s_{k-1} \cdot \beta_3 + \frac{\|\nabla f_k(w_k)\|^2}{\sqrt{\hat{v}_k} + \epsilon} \cdot (1-\beta_3) \tag{8}$$

while also using Equations (2) to compute the Adam direction.

3.1 Reduce Computational Overhead

Line search methods-of any kind-have a very impractical design issue, they need to evaluate the loss (and often the gradient) of the current mini-batch at every iteration. To address this, modify the method to perform a line search regularly when a high rate of change of the step size is detected, and less regularly otherwise. Keep 2 exponential moving averages of the step size η_k:

$$\bar{\eta}_k(\beta) = \beta \bar{\eta}_{k-1} + (1-\beta) \cdot \eta_{k-1} \tag{9}$$

The average rate of change:

$$\bar{r}_k = \begin{cases} r_k, & r_k \geq 1 \\ r_k^{-1}, & \text{otherwise} \end{cases} \quad \text{where} \quad r_k = \frac{\bar{\eta}_k(0.9)}{\bar{\eta}_k(0.99)} \tag{10}$$

Perform line search with frequency:

$$L_k = \frac{1}{\bar{r}_k - 1} \tag{11}$$

This inner modification of the algorithm reduces the computational requirements of the line search method from 30% (naive line search) to approximately $\sim 3\%$ (SaLSa) for longer runs.

4 Setup & Results

4.1 Setup summary

- **Candidate optimizers:** ADAM, SGD; variants with SLS and SaLSa.
- **Datasets / models:** GLUE (BERT fine-tuning: SST2, MRPC, MNLI...), ResNet on CIFAR-10/100, ResNet50 on ImageNet.
- **Implementation:** experiments run 5×; reported means ± SE. Line-search frequency adaptation reduces overhead to $\approx 3\%$.

4.2 Visual Comparison

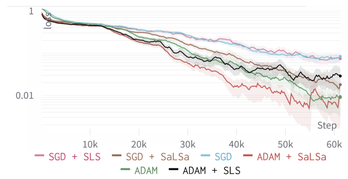

Figure 4: (a) MNLI — training loss

Takeaway: ADAM+SaLSa reduces training loss faster and to a lower final value than ADAM and ADAM+SLS.

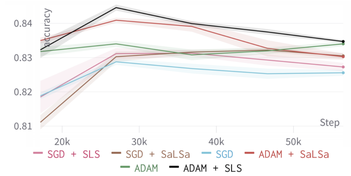

Figure 5: (b) MNLI — validation accuracy

Takeaway: Line-search variants converge faster; SaLSa is stable across runs.

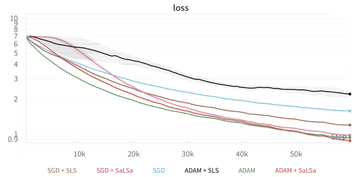

Figure 6: (c) ImageNet — training loss

Takeaway: ADAM+SaLSa attains lower final training loss than ADAM and ADAM+SLS; SLS shows instability.

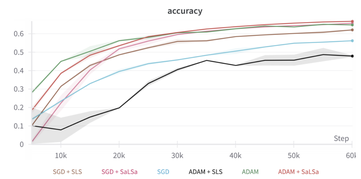

Figure 7: (d) ImageNet — validation accuracy

Takeaway: SaLSa yields best or near-best peak accuracy and more stable curves than SLS.

4.3 Accuracy Examples

Dataset	ADAM	ADAM + SLS	ADAM + SaLSa
MNLI	0.8340	0.8446	**0.8409**
MRPC	0.8279	0.8667	**0.8745**
ImageNet (R50)	0.6486	0.4836	**0.6684**

4.4 Selected Final Training Losses

Dataset	ADAM	ADAM + SLS	ADAM + SaLSa
MNLI	0.009567	0.03713	**0.005867**
MRPC	0.01312	0.007298	**0.003126**
ImageNet (R50)	0.9122	2.036	**0.8339**

4.5 Conclusion

- SaLSa removes manual LR schedules: typically matches or surpasses tuned baselines (ADAM/SGD) without per-experiment LR tuning.
- SaLSa attains lower final loss and improved stability (notably on large-scale tasks such as ImageNet and Transformer fine-tuning).
- Computational cost: SaLSa $\approx 3\%$ extra compute vs naive line-search $\approx 30\%$.
- Empirical result: across experiments SaLSa yields better average final loss and often better peak accuracy (see selected numbers above).